

Chapter 4 Basic Arithmetic Operators

- Addition and Subtraction

- ▶ 4.1 Addition of the computer
- ▶ 4.2 Ripple Carry Adder (RCA)
- ▶ 4.3 Carry-forward adder (CLA)
- ▶ 4.4 Block Ripple Carry Adder (BRCA)
- ▶ 4.5 Multi-layer structure of fast adder design
- ▶ 4.6 Subtraction and addition and subtraction device design
- ▶ 4.7 Binary encoded decimal (BCD) adder

4.1 Addition of computers

- ▶ Addition is the most basic operation in computers.
 - ▶ Subtraction, multiplication, or division can all be done using additive operations.
- ▶ The hardware circuit that implements addition is called the **adder**, which is the most basic component of computer arithmetic.
- ▶ Carry is one of the most important factors to consider in addition operations, and all circuits that handle addition are designed to focus on how to solve the **carry problem**.

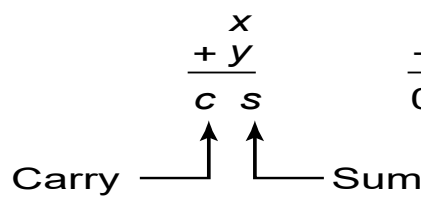
4.2 Ripple carry adder (RCA)

- ▶ A single-digit adder
- ▶ A ripple carry adder consisting of a single element total adder
- ▶ Time delay for the ripple carry adder

4.2.1 A single-digit adder

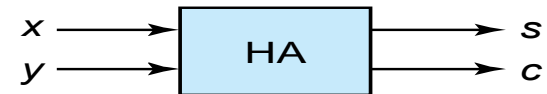
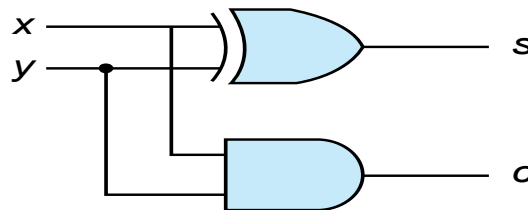
► half adder(HA) :

- Consider the sum of the two input bits and the carry value.
- The time delay of 1 logic gate can be completed.



$\begin{array}{r} 0 \\ + 0 \\ \hline 0 \quad 0 \end{array}$	$\begin{array}{r} 0 \\ + 1 \\ \hline 0 \quad 1 \end{array}$	$\begin{array}{r} 1 \\ + 0 \\ \hline 0 \quad 1 \end{array}$	$\begin{array}{r} 1 \\ + 1 \\ \hline 1 \quad 0 \end{array}$
---	---	---	---

x	y	Carry c	Sum s
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0



Truth table and circuit diagram of the unit element half adder

4.2.1 A single-digit adder

► full adder(FA) :

- Consider 2 input bits.
- Consider the carry value generated by the addition of the previous digit.
- It needs to pass through the delay time of 3 logic gates.

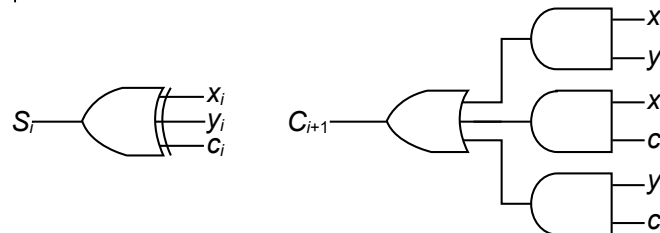
c_i	x_i	y_i	c_{i+1}	s_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$x_i y_i$	00	01	11	10
c_i				
0		1		1
1	1		1	

$$s_i = x_i \oplus y_i \oplus c_i$$

$x_i y_i$	00	01	11	10
c_i				
0			1	
1		1	1	1

$$c_{i+1} = x_i y_i + x_i c_i + y_i c_i$$

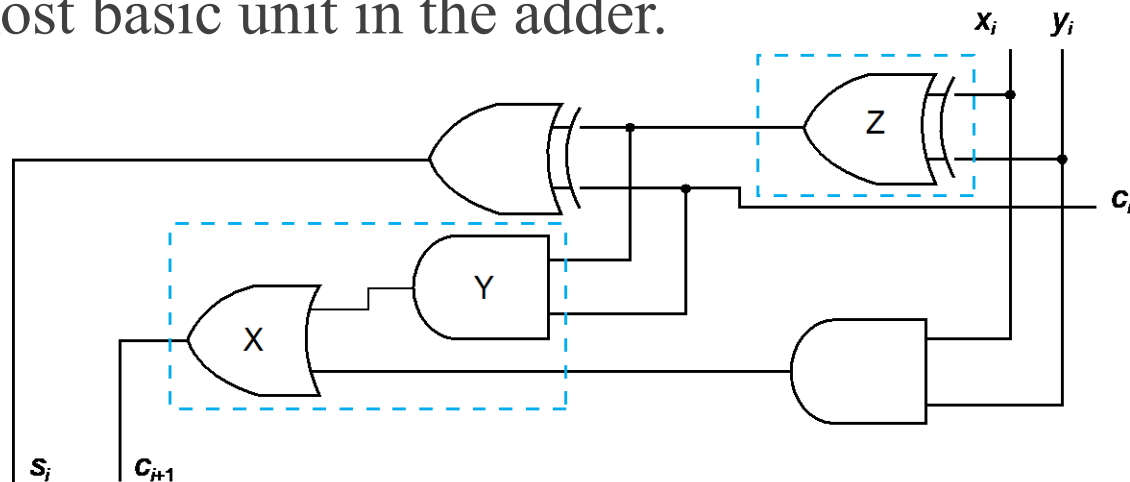


The circuit diagram drawn by the sum of the original total adder and the carry logic equation

4.2.1 A single-digit adder

► full adder(FA) :

- The logic formula obtained by the full adder can be further simplified to generate a simplified logic circuit.
- After simplification, the circuit only takes 2 logic gates to complete a single operation.
- The most basic unit in the adder.



simplified unit element full adder circuit

4.2 Ripple carry adder (RCA)

- ▶ A single-digit adder
- ▶ A ripple carry adder consisting of a single element total adder
- ▶ Time delay for the ripple carry adder

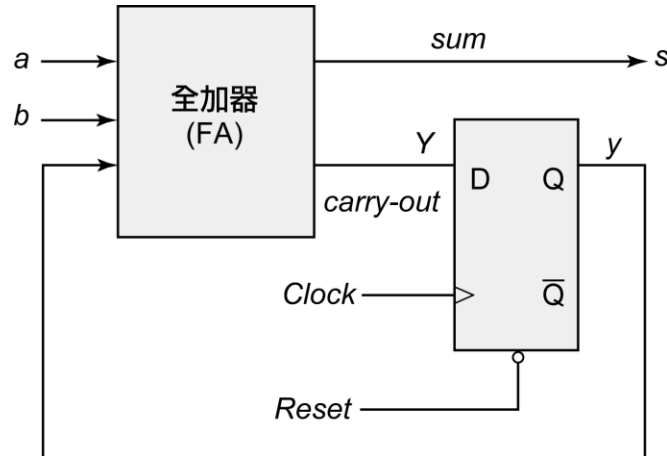
4.2.2 A ripple carry adder consisting of a single element total adder

► sequential adder :

- From the lowest digit (also known as least significant bit, or LSB) to the highest digit (also known as most significant bit, or MSB) digit by digit, input one digit after another and complete the addition design.
- Only 3 input bits and 2 output bits are involved each time, and how many digits there are, how many operations must be performed.
- It can be done with a full adder and a 1-bit carry register.
- It takes $3n$ gate delays to complete n -bit number addition.

4.2.2 A ripple carry adder consisting of a single element total adder

► sequential adder :



Clock	a	b	y (carry-in)	s (sum)	Y (carry-out)
1	1	1	0 (預設)	0	1
2	0	1	1	0	1
3	0	0	1	1	0
4	1	0	0	1	0

Examples of sequential adder circuits
and computational processes

4.2.2 A ripple carry adder consisting of a single element total adder

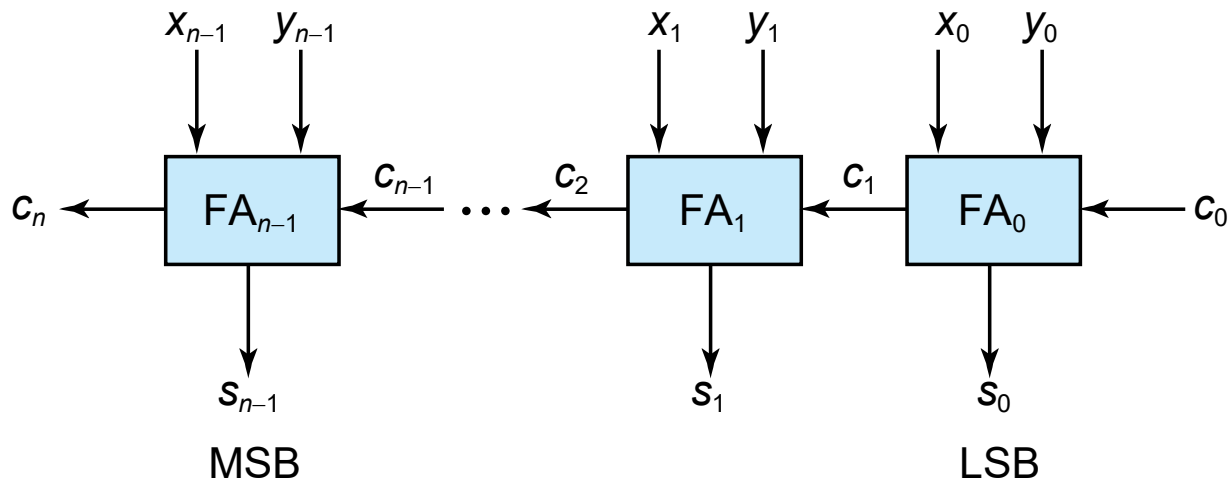
► parallel adder :

- Also known as ripple carry adder (Ripple Carry Adder , RCA) ◦
- To calculate the sum result of two n -bit numbers, you need to enter two numbers with a total of $2n$ bits at the same time.
- The addition of each bit needs to be done using a full adder, so the addition of n bits requires the use of n full adders.

4.2.2 A ripple carry adder consisting of a single element total adder

► parallel adder :

► *n bit parallel RCA adder structure :*



4.2 Ripple carry adder (RCA)

- ▶ A single-digit adder
- ▶ A ripple carry adder consisting of a single element total adder
- ▶ Time delay for the ripple carry adder

4.2.3 Time delay for the ripple carry adder

- ▶ Consider RCA for n -bits :
 - ▶ The delay time of one logic gate completes the AND and XOR operations of all pairs of x_i and y_i .
 - ▶ The time of the 2 logic gates is then used to calculate the carry-out and pass it to the higher digit.
 - ▶ The total delay of the parallel ripple adder circuit should be the delay time of $2n+1$ logic gates.

4.3 Carry forward adder (CLA)

- ▶ Analysis of carry generation conditions
- ▶ The adder design that precomputes the carry

4.3.1 Analysis of carry generation conditions

► Carry and sum operations :

- For the full adder FA_i, which is responsible for the ith digit, the Boolean algebraic formula for the sum value S_i and the carry value C_{i+1} produced by its operation is :

$$S_i = A_i \oplus B_i \oplus C_i$$

$$C_{i+1} = A_i B_i + (A_i \oplus B_i) C_i$$

- The rounding value of each bit (C_{i+1}) must be determined by the rounding value (C_i) of the lower bit.
 - The main key to the RCA adder generating latency.

4.3.1 Analysis of carry generation conditions

- ▶ Carry generation and carry pass :

- ▶ Generate(G_i)

- ▶ $G_i = A_i B_i$

- ▶ Propagate(P_i)

- ▶ $P_i = A_i \oplus B_i$

4.3.1 Analysis of carry generation conditions

► Carry generation and carry pass :

- Substituting G_i and P_i into the C_{i+1} and S_i Boolean algebraic formulas of the original total adder can be rewritten as:

$$\begin{array}{ll} S_i = A_i \oplus B_i \oplus C_i & P_i = A_i \oplus B_i \\ C_{i+1} = A_i B_i + (A_i \oplus B_i) C_i & G_i = A_i B_i \end{array}$$

$$S_i = A_i \oplus B_i \oplus C_i$$

$$= P_i \oplus C_i$$

$$C_{i+1} = A_i B_i + (A_i \oplus B_i) C_i$$

$$= G_i + P_i C_i$$

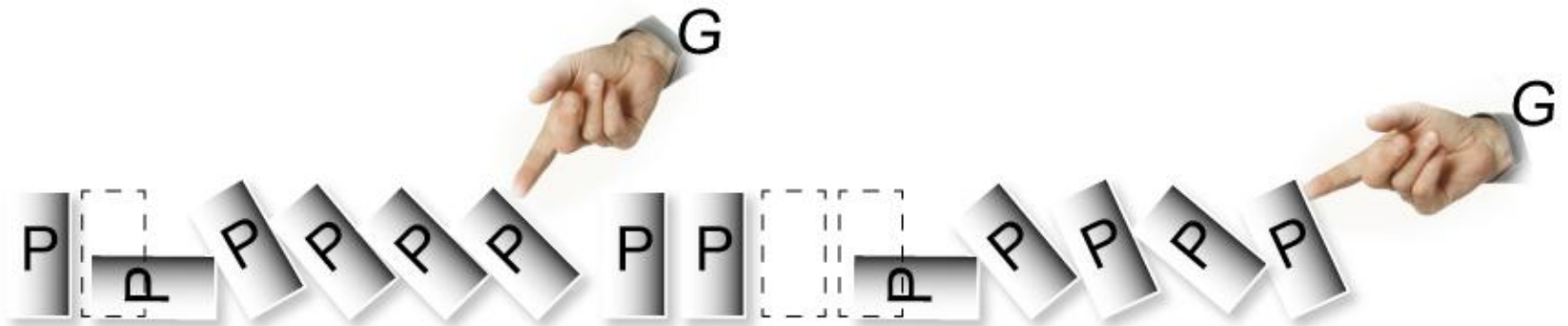
4.3.1 Analysis of carry generation conditions

- Carry generation and carry pass :
- By using iterative techniques to untangle recursive relationships, the purpose of carry forward can be achieved :

$$\begin{aligned}C_1 &= G_0 + P_0 C_0 \\C_2 &= G_1 + P_1 C_1 && \downarrow \\&= G_1 + P_1 (G_0 + P_0 C_0) && \downarrow \\&= G_1 + P_1 G_0 + P_1 P_0 C_0 \\C_3 &= G_2 + P_2 C_2 \\&= G_2 + P_2 (G_1 + P_1 G_0 + P_1 P_0 C_0) \\&= G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0 \\C_4 &= G_3 + P_3 C_3 && \downarrow \\&= G_3 + P_3 (G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_0) \\&= G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_0 \\&\vdots && \vdots \\C_{i+1} &= G_i + P_i C_i \\&= G_i + P_i G_{i-1} + P_i P_{i-1} G_{i-2} + \dots + P_i P_{i-1} \dots P_1 G_0 + P_i P_{i-1} \dots P_1 P_0 C_0\end{aligned}$$

4.3.1 Analysis of carry generation conditions

- *The definitions of G and P values are very abstract, but in fact, as long as you use a little imagination and think of it as playing a **domino game**, it is not difficult to understand the mystery of it :*



4.3.1 Analysis of carry generation conditions

- ▶ Carry generation (G): If the value of the two input bits of the i th digit is 1 ($G_i = A_i B_i = 1$), then a carry will definitely be generated here; It's like we're knocking a domino forward directly from this place, and it's right in front of it ($i+1$) and it's going to knock it down if there's a domino.
- ▶ Carry Transfer (P) If one of the two input bits happens to be 1 ($P_i = A_i \vee B_i = 1$), then as long as the carry input value is also 1, the carry output can be generated; It's like putting a domino in this position, when the back is turned upside down, you will be pushed forward!

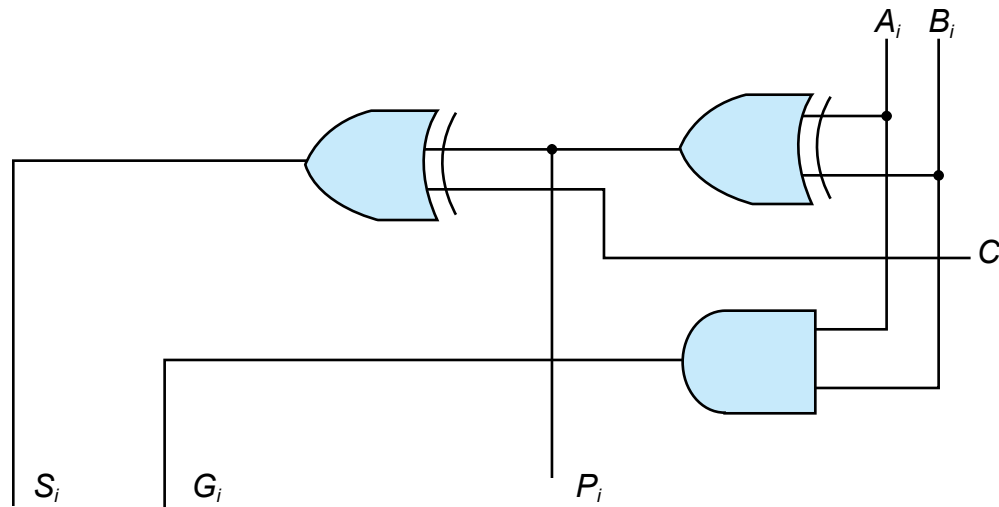
4.3 Carry forward adder (CLA)

- ▶ Analysis of carry generation conditions
- ▶ The adder design that precomputes the carry

4.3.2 The adder design that precomputes the carry

- ▶ The CLA part of each individual bit is fully added circuit
- ▶ partially full adder (PFA)

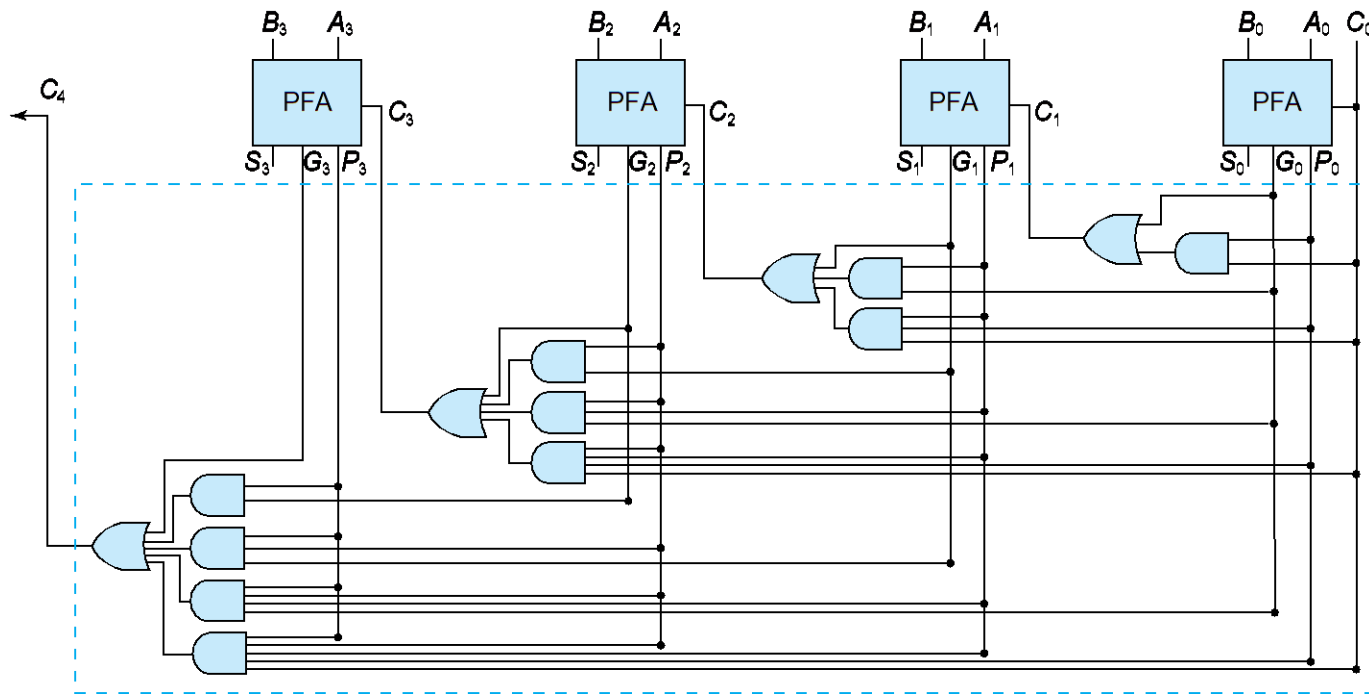
▶ The local total adder of each individual bit "and" S_i can be calculated ◦



Partial full adder circuit

4.3.2 The adder design that precomputes the carry

- Round-up look-ahead adder per unit length :
 - A partial all-adder circuit of all bits is combined with a carry forward logic circuit to form a complete carry forward adder.



4-bit carry forward adder circuit

4.3.2 The adder design that precomputes the carry

- ▶ Latency analysis of CLA :
 - ▶ After a delay of one logic gate, all G_i and P_i can be generated.
 - ▶ After 2 logic gates are delayed, all C_i can be obtained.
 - ▶ After 1 logic gate delay, all S_i can be obtained.
 - ▶ The total delay of each carry forward adder is fixed at 4 logic gate delay times, regardless of its length (number of bits).

4.4 Block ripple carry adder (BRCA)

- ▶ CLA adder :

- ▶ advantage :

- ▶ Improved RCA adder carry delay issue.

- ▶ disadvantage :

- ▶ Improves the complexity of carry circuits.

- ▶ The carry forward-looking logic circuit will increase more and more rapidly as the number of bits in the full adder increases.

- ▶ It is not possible to design a full CLA adder for any number of bits required.

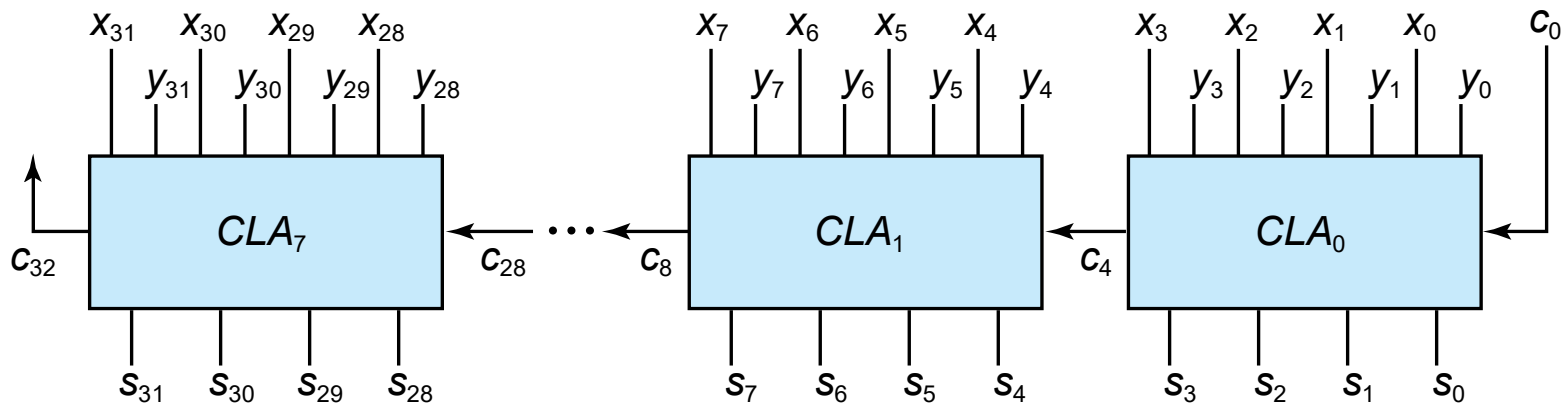
4.4 Block ripple carry adder (BRCA)

- ▶ Block ripple structure
- ▶ Latency analysis of the block ripple adder
- ▶ Trade space for time - carry selection adder
(Carry-Select Adder , CSA)

4.4.1 Block ripple structure

► block ripple carry adder(BRCA) :

- The basic CLA addition blocks of fixed bits are connected in a ripple carrying manner.



Block ripple carry adder circuit

4.4 Block ripple carry adder (BRCA)

- ▶ Block ripple structure
- ▶ Latency analysis of the block ripple adder
- ▶ Trade space for time - carry selection adder
(Carry-Select Adder , CSA)

4.4.2 Latency analysis of the block ripple adder

► Latency analysis :

- At first, all bits generate a common logic gate delay (generating G_i and P_i).
- The carry operation for each CLA addition block results in 2 gate delays.
- After the CLA addition block representing the highest bit completes the rounding operation, it needs to add one more logic gate delay to calculate the sum of the highest digits.

4.4.2 Latency analysis of the block ripple adder

► Total delay formula for a ripple carry adder circuit :

$$d = 2 \left\lceil \frac{n}{m} \right\rceil + 2$$

d : Ripple carry adder delay
 m : CLA adds block bits
 n : Additive operation bits

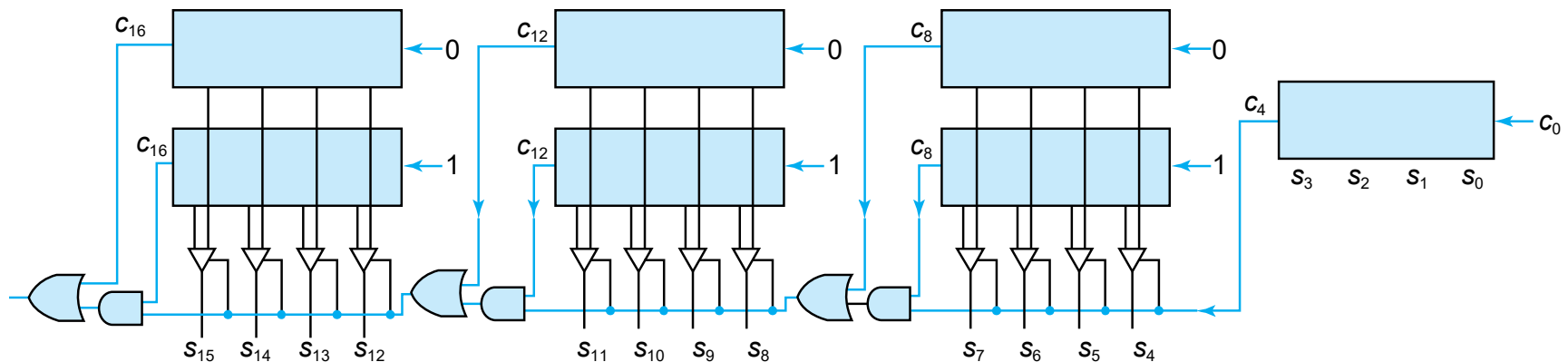
4.4 Block ripple carry adder (BRCA)

- ▶ Block ripple structure
- ▶ Latency analysis of the block ripple adder
- ▶ Trade space for time - carry selection adder
(Carry-Select Adder , CSA)

4.4.3 Trade space for time - carry selection adder (Carry-Select Adder , CSA)

► Carry selection adder :

- The total latency is reduced to $\left\lceil \frac{n}{m} \right\rceil + 3$



The small triangles below each pair of CLA blocks in the figure are multiplexers, and they choose the output value generated by one of the blocks based on the carry input value of the block.

4.5 Quick adder design with multi-layer structure

- ▶ Changing the concept of design - parallel processing
- ▶ An adder that implements a double-layer structure
- ▶ Re-extension of the fast adder - a three-layer structure
- ▶ Performance analysis of the fast adder

4.5.1 Changing the concept of design - parallel processing

► Parallel processing :

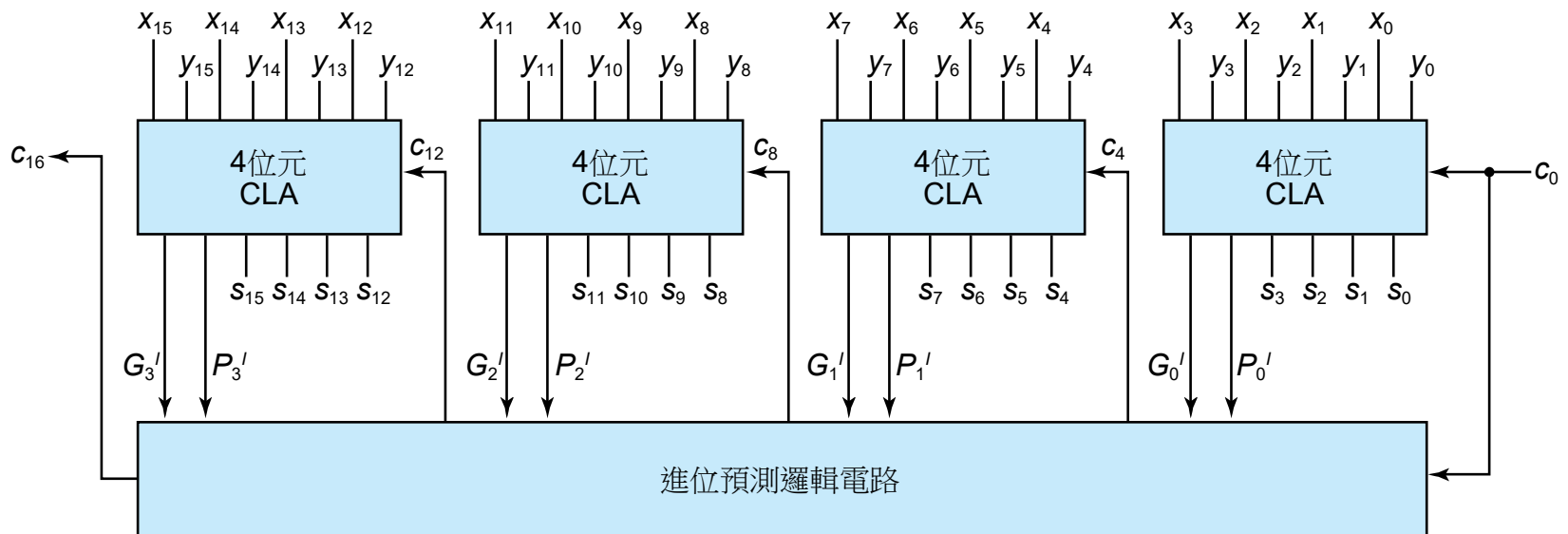
- In terms of the design of the adder, when we have a high-performance 4-bit CLA adder unit, if we want to form a higher-digit adder, such as a 16-bit adder, then in addition to directly connecting into a block ripple carry adder with 4 4-bit CLA adders, we can also use the concept of generate and propagate to complete the calculation of all CLA carry inputs in a faster way. Then let the 4 CLAs start calculating the final sum at the same time.

4.5 Quick adder design with multi-layer structure

- ▶ Changing the concept of design - parallel processing
- ▶ An adder that implements a double-layer structure
- ▶ Re-extension of the fast adder - a three-layer structure
- ▶ Performance analysis of the fast adder

4.5.2 An adder that implements a double-layer structure

- The concept of two-dimensional combination :
- Utilize 2D to implement a 16-bit fast adder.



Double-layer carry forward-looking adder circuit

4.5.2 An adder that implements a double-layer structure

- ▶ Calculation method for carry forward-looking :
 - ▶ The 4 CLAs are :
 - ▶ CLA_0 、 CLA_1 、 CLA_2 、 CLA_3
 - ▶ Its carry inputs are :
 - ▶ C_0 、 C_4 、 C_8 、 C_{12}
 - ▶ Its carry output is respectively :
 - ▶ C_4 、 C_8 、 C_{12} 、 C_{16}

4.5.2 An adder that implements a double-layer structure

► Calculation method for carry forward-looking :

$$C_4 = G_0^I + P_0^I C_0$$

$$\begin{aligned} C_8 &= G_1^I + P_1^I C_4 \\ &= G_1^I + P_1^I (G_0^I + P_0^I C_0) \\ &= G_1^I + P_1^I G_0^I + P_1^I P_0^I C_0 \end{aligned}$$

$$\begin{aligned} C_{12} &= G_2^I + P_2^I C_8 \\ &= G_2^I + P_2^I (G_1^I + P_1^I G_0^I + P_1^I P_0^I C_0) \\ &= G_2^I + P_2^I G_1^I + P_2^I P_1^I G_0^I + P_2^I P_1^I P_0^I C_0 \end{aligned}$$

$$\begin{aligned} C_{16} &= G_3^I + P_3^I C_{12} \\ &= G_3^I + P_3^I (G_2^I + P_2^I G_1^I + P_2^I P_1^I G_0^I + P_2^I P_1^I P_0^I C_0) \\ &= G_3^I + P_3^I G_2^I + P_3^I P_2^I G_1^I + P_3^I P_2^I P_1^I G_0^I + P_3^I P_2^I P_1^I P_0^I C_0 \end{aligned}$$

4.5.2 An adder that implements a double-layer structure

► Calculation method for carry forward-looking :

$$G_i^I = G_{4i+3} + P_{4i+3}G_{4i+2} + P_{4i+3}P_{4i+2}G_{4i+1} + P_{4i+3}P_{4i+2}P_{4i+1}G_{4i}$$

$$P_i^I = P_{4i+3}P_{4i+2}P_{4i+1}P_{4i}$$

4.5.2 An adder that implements a double-layer structure

► Operation of the double-layer adder :

- Step 1 : *After entering 16 bits each of the numbers A and B, the digits are first calculated G_i and P_i respectively.*
- Step 2 : Everyone G_j^I and P_j^I ($i=0\sim15$) Four first-level forward logic circuits are passed in and then calculated separately G_j^I and P_j^I ($j=0\sim3$) ◦
- Step 3 : 4 groups G_j^I and P_j^I After passing in the second-level carry forward logic circuit, with the input of C0, C4, C8, C12, and C16 are calculated.

4.5.2 An adder that implements a double-layer structure

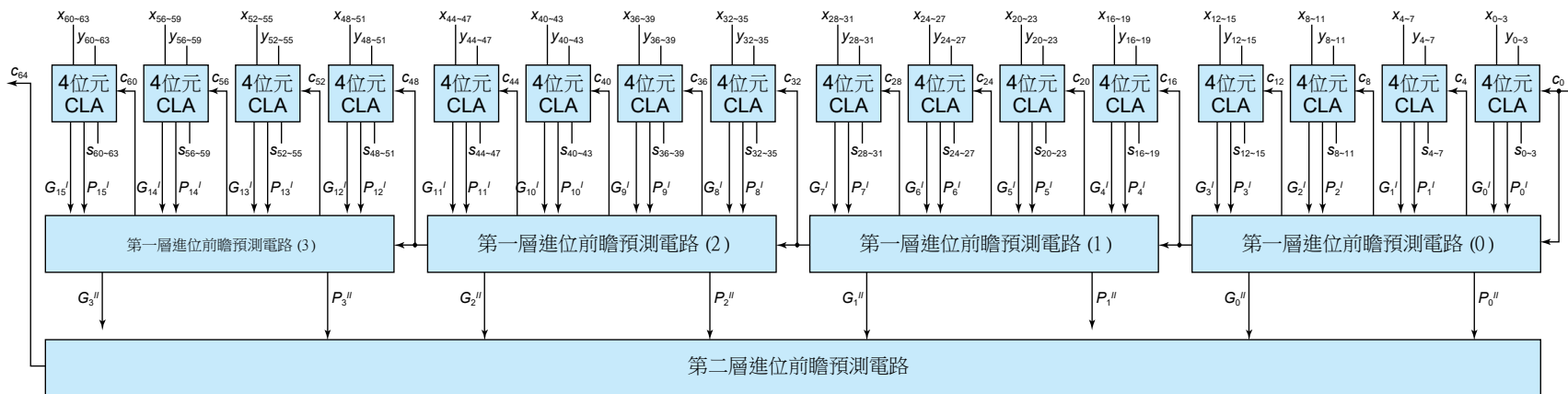
- ▶ Step 4 : C_0 、 C_4 、 C_8 、 C_{12} Input 4 first-level forward logic circuits respectively , With all G_i and P_i ($i = 0 \sim 15$), to get all C_i ($i = 0 \sim 15$).
- ▶ Step 5 : all of the C_i ($i = 0 \sim 15$) Returns the top partial full adder and calculates the final sum S .

4.5 Quick adder design with multi-layer structure

- ▶ Changing the concept of design - parallel processing
- ▶ An adder that implements a double-layer structure
- ▶ Re-extension of the fast adder - a three-layer structure
- ▶ Performance analysis of the fast adder

4.5.3 Re-extension of the fast adder - a three-layer structure

► Three-layer 64-bit adder :



Three-layer carry forward-
looking adder circuit

4.5.3 Reextension of the fast adder - a three-layer structure

► Operation process :

- Step 1 : *After all the bits of A and B are entered, G_i and P_i are calculated respectively.*
- Step 2 : The G_i and P_i ($i=0\sim63$) of each digit are passed into 16 first-order forward logic circuits, and then G_j^I and P_j^I ($j=0\sim15$) are calculated respectively.
- Step 3 : After passing in 4 second-level carry forward logic circuits in 16 groups G_j^I and P_j^I , G_k^{II} and P_k^{II} ($k=0\sim3$) are calculated respectively).

4.5.3 Re-extension of the fast adder - a three-layer structure

- ▶ Step 4 : After 4 groups G_j^I and P_j^I are passed into the third-level carry-forward logic circuit, with the input of C0, C16, C32, C48, C64 are calculated.
- ▶ Step 5 : $C0$ 、 $C16$ 、 $C32$ 、 $C48$,Input 4 second-level forward logic circuits respectively , Combine all G_k^{II} and P_k^{II} to get all $C4j$ ($j = 0\sim 15$).

4.5.3 Re-extension of the fast adder - a three-layer structure

- ▶ Step 6 : $C4j(j = 0 \sim 15)$ inputs 16 first-order forward-looking logic circuits respectively, and with all G_i and P_i , all $C_i(i = 0 \sim 63)$ are obtained.
- ▶ Step 7 : All $C_i(i = 0 \sim 63)$ return the top 64 partial total adders to calculate the final sum S .

4.5 Quick adder design with multi-layer structure

- ▶ Changing the concept of design - parallel processing
- ▶ An adder that implements a double-layer structure
- ▶ Re-extension of the fast adder - a three-layer structure
- ▶ Performance analysis of the fast adder

4.5.4 Performance analysis of the fast adder

► 16-bit fast adder performance analysis :

Item	Compute content	Required gate delay
Step 1	Calculate G_i and P_i	1
Step 2	Calculate G_{jI} and P_{jI}	2
Step 3	Calculate C_4 、 C_8 、 C_{12} 、 C_{16}	2
Step 4	Calculate the C_i for all bits	2
Step 5	Calculate the final sum S	1
Sum	Complete the addition of 16 bits	8

4.5.4 Performance analysis of the fast adder

► 64-bit fast adder performance analysis :

Item	Compute content	Required gate delay
Step 1	Calculate G_i and P_i	1
Step 2	Calculate G_j^I and P_j^I	2
Step 3	Calculate G_k^{II} and P_k^{II}	2
Step 4	Calculate C_{16} 、 C_{32} 、 C_{48} 、 C_{64}	2
Step 5	Calculate all of them C_{4j} , $j = 0\sim 15$	2
Step 6	Count all bits C_i	2
Step 7	Calculate the final sum S	1
Sum	Complete the addition of 64 bits	12

4.6 Subtraction and addition and subtraction device design

- ▶ Line sharing and addition and subtraction options
- ▶ overflow treatment

4.6.1 Line sharing and addition and subtraction options

► Addition/subtraction machine :

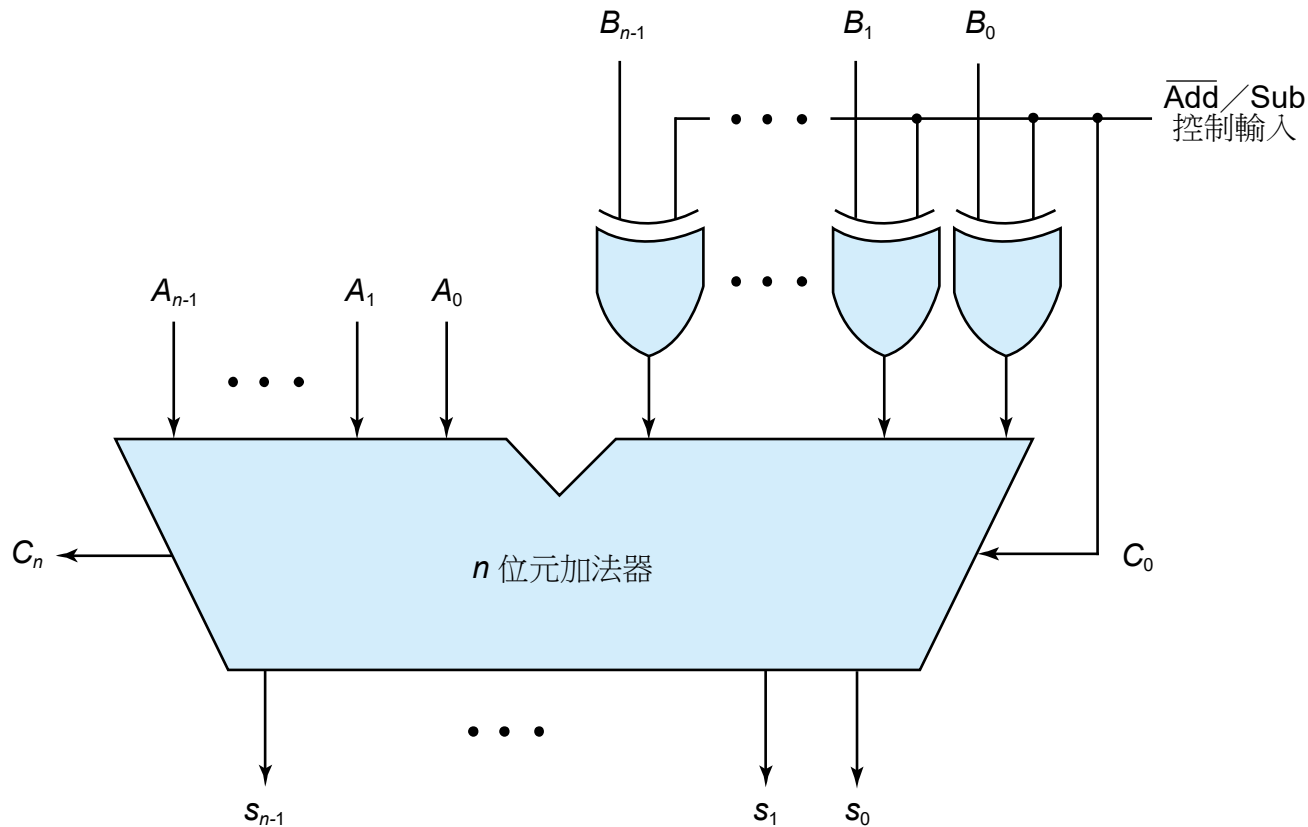
► The design of the subtractor integrated into the adder ◦

► principle :

► For any addition operation, such as $A+B$, it can be regarded as an addition operation $A+B+0$; for any subtraction operation $A-B$, it can be regarded as an addition operation with the number $A+(B)$, and if (B) is expressed as $(\sim B)+1$ by a binary complement, then the subtraction operation $A-B$ can be regarded as the addition operation $A+(\sim B)+1$.

4.6.1 Line sharing and addition and subtraction options

► Addition/Subtractor Circuit :



4.6 Subtraction and addition and subtraction device design

- ▶ Line sharing and addition and subtraction options
- ▶ overflow treatment

4.6.2 overflow treatment

- ▶ Detection rules for addition and subtraction overflow phenomenon :
 - ▶ In the process of addition and subtraction, if the carry input value of the symbol bit (usually the highest bit) is equal to the carry output value of the symbol bit, there is no overflow phenomenon; If the carry input value of the symbol bit is not equal to the carry output value of the symbol bit, an overflow phenomenon occurs.
 - ▶ There are usually four situations in which overflow occurs in addition and subtraction instruments: positive + positive, negative + negative, positive-negative, and negative-positive.

4.6.2 overflow treatment

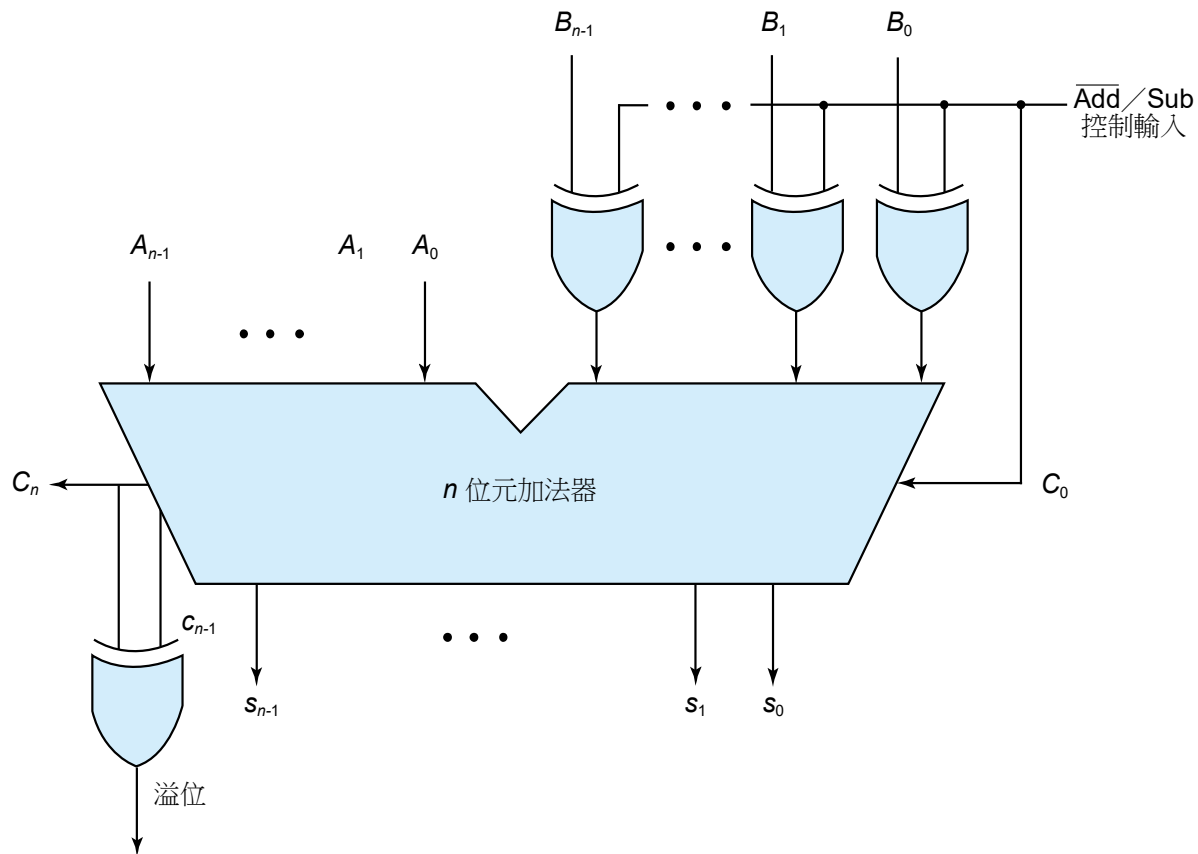
► The detection of additive and subtractor overflow phenomena can be done by using XOR logic gates.

- C_{in} : The rounded input value of the symbol bit.
- C_{out} : carry output value.

C_{in}	C_{out}	$C_{in} \text{ XOR } C_{out}$
0	0	0
0	1	1
1	0	1
1	1	0

4.6.2 overflow treatment

► Adder circuit with overflow detection :



4.7 Binary encoded decimal (BCD) adder

- ▶ Binary encoded decimal (BCD) notation :
 - ▶ Each digit of the decimal number is encoded in binary form.

Decimal digit	BCD code
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

decimal numbers and their corresponding binary codes

4.7 Binary encoded decimal (BCD) adder

- ▶ Binary encoding decimal addition problems :
 - ▶ The addition structure of two binary encoded decimal numbers is more complex, in fact the sum of each digit may exceed 9, in which case the number correction action is necessary.

4.7 Binary encoded decimal (BCD) adder

► Solutions to addition problems :

► Two situations where you need to correct the action :

► The sum of the two four-digit numbers exceeds, but there is no rounding

► The sum of the two four-digit numbers exceeds 15, resulting in a rounding phenomenon.

$$\begin{array}{r} X \quad \quad 0111 \quad \quad 7 \\ + Y \quad + 0101 \quad + 5 \\ \hline Z \quad \quad 1100 \quad \quad 12 \\ \quad \quad + 0110 \\ \hline \end{array}$$

Carry $\rightarrow$ $\underbrace{10010}_{S=2}$

$$\begin{array}{r} X \quad \quad 1000 \quad \quad 8 \\ + Y \quad + 1001 \quad + 9 \\ \hline Z \quad \quad 10001 \quad \quad 17 \\ \quad \quad + 0110 \\ \hline \end{array}$$

Carry $\rightarrow$ $\underbrace{10111}_{S=7}$

4.7 Binary encoded decimal (BCD) adder

- Solutions to addition problems :

- Number correction principle :

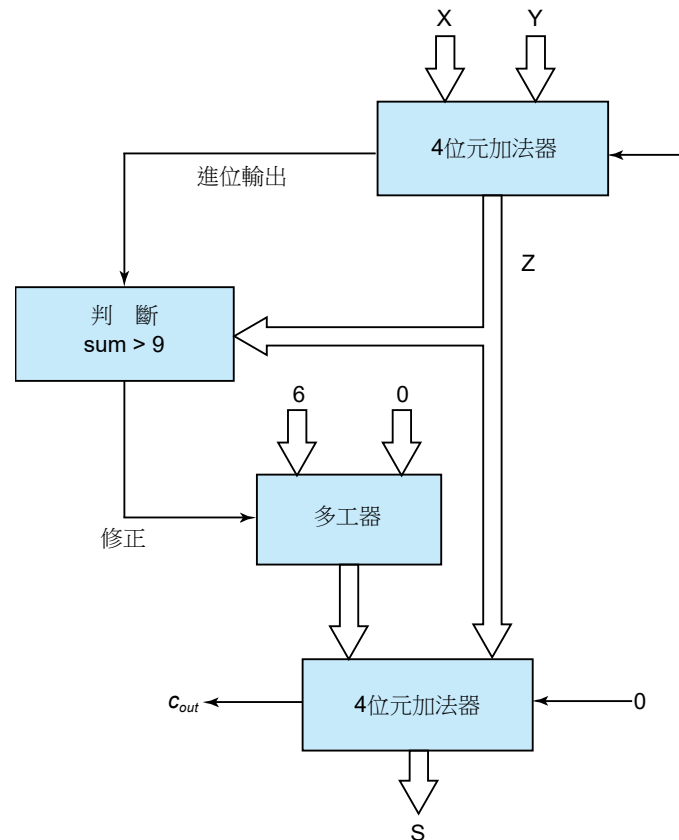
$$Z = X + Y$$

If , $Z \leq 9$ then $S = Z$ and rounded to 0;

if $Z > 9$, then $S = Z + 6$ and rounded to 1;

4.7 Binary encoded decimal (BCD) adder

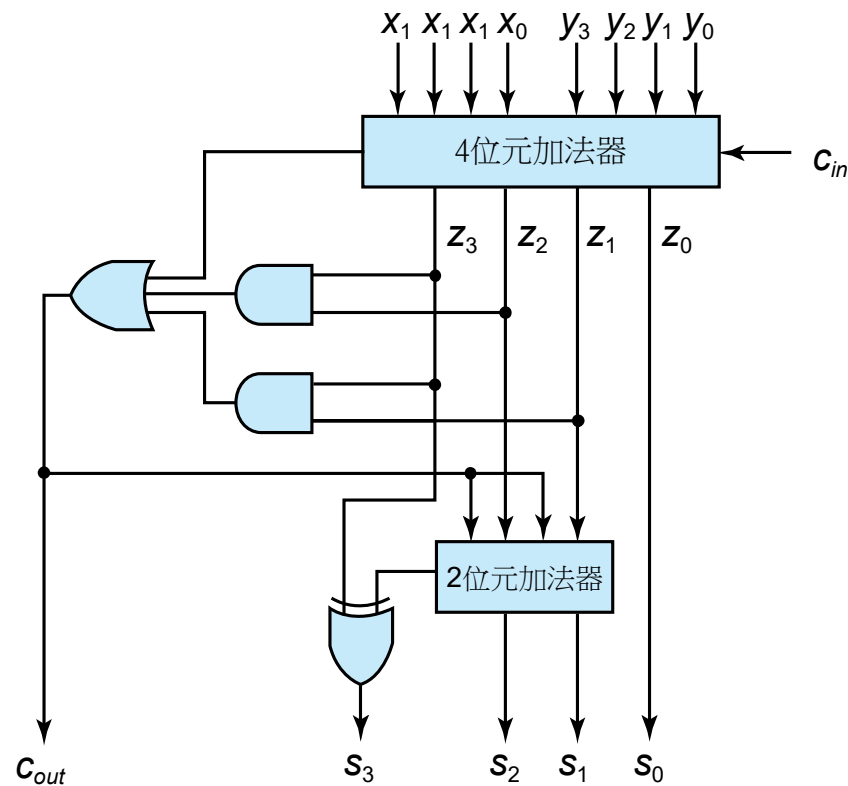
► Construct a unit number BCD adder :



Block diagram of the unit number BCD adder

4.7 Binary encoded decimal (BCD) adder

► Construct a unit number BCD adder :



One-digit binary encoded decimal adder circuit